

Odds-and-Evens Game Implementation

1. Problem Analysis

The assignment requires implementing a tournament-style Odds-and-Evens game for $n = 2^m$ players using the Go language. The primary challenge from a concurrent perspective is managing the synchronization and execution of multiple games simultaneously without relying on shared memory.

The key concurrent aspects are:

- The tournament must proceed in m stages, where all matches in a current round must finish before the next round can begin.
- Matches within a single round are independent and should run concurrently.
- The **Referee** coordinates players and collect results safely using message passing instead of shared state.

2. General Strategy

The implementation follows a **centralized coordination strategy** where a **Referee** manages the tournament and individual **Player** goroutines.

1. At the start, n player goroutines are spawned. Each player runs a persistent loop, blocked on a channel, waiting for a signal to randomly generate a move (throw either 1 or 2).
2. The **Referee** iterates through m rounds. In each round, players are paired off (player 0 vs player 1, player 2 vs player 3, etc.). Each match is run in its own goroutine.
3. The **Referee** uses a buffered **Player** channel to collect the winners of each match. Once the channel collected all the winners of the current round, the **Referee** updates the player list for the next round.
4. This cycle continues until only one undefeated player remains, at which point the **Referee** declares them the tournament winner.

To adhere to the “no shared memory” requirement, all coordination is handled via Go channels:

- The Referee sends a boolean signal to a player’s **in** channel to request a move.
- The player sends back a **Move** struct (containing their ID and chosen value) back to the Referee through an **out** channel.
- Communication is also used for termination. When a player is eliminated, the Referee closes their **in** channel. The player goroutine detects this closure and exits its loop.

Smart Home Alarm System Implementation (Pekko/Scala)

1. Problem Analysis

Implementing a smart home alarm system requires a concurrent architecture capable of handling multiple asynchronous interactions. The system must process events from a network of sensors and user inputs from a keypad while managing time-sensitive state transitions.

The key concurrent and architectural challenges are that:

- The **AlarmControlUnit** must receive inputs from sensors and a keypad without those peripheral components needing to know the system's current internal state.
- The transition from **disarmed** to **armed** state is handled by an **exitDelay** state. In such a state the user must be able to cancel the arming process by entering the correct PIN before a timeout triggers the full transition.
- When in the **armed** state, the system must monitor active zones to transition to an **entryDelay** state upon intrusion detection.
- The system must ensure that if a correct PIN is received during the **entryDelay** state, then the timeout is cancelled and therefore the alarm won't be triggered afterward.
- Once in the **alarm** state, the system must remain responsive to keypad inputs as the only mean to silence the siren and transition back to a **disarmed** state.

2. General Strategy

The implementation leverages the **Actor Model** via Pekko, treating each component as an isolated entity that communicates exclusively through asynchronous message passing.

Implementation Details

- The **AlarmControlUnitActor** logic is driven by a strictly defined set of inputs within the **AlarmControlUnitInput** protocol. This ensures that the actor only processes relevant domain events:
 - **SensorTriggered**: Carries sensor metadata to check against active zones.
 - **ArmRequest**: Initiates the arming sequence with a PIN and a specific set of zones.
 - **PinEntered**: Used for disarming the system or cancelling delay states.
 - **Timeouts (ExitTimeout/EntryTimeout)**: Internal signals that trigger automatic state transitions after a delay.

- The actor system follows a **centralized topology** organized into a tree managed by the `AlarmSystemGuardian`. All domain decisions flow through the `AlarmControlUnitActor`. Peripheral actors (`KeypadActor`, `SensorActor`, `SirenActor`) only send commands to it or receive them from it, never communicating with each other directly. The `AlarmSystemGuardian` initializes the `AlarmControlUnitActor`, `SirenActor`, `KeypadActor`, and spawns a specific `SensorActor` for every physical sensor in the configuration. Each actor has a single, well-defined responsibility:
 - **KeypadActor**: Accumulates digit presses into an internal string buffer. On `PressEnter`, sends `PinEntered(buffer)` to the `AlarmControlUnitActor` (used for disarming or cancelling delays). On `PressEnterWithZones(zones)`, sends `ArmRequest(buffer, zones)` (used to initiate arming). Both flush the buffer on send. `PressClear` resets the buffer without sending.
 - **SensorActor**: A stateless forwarder, one instance per physical sensor. On `DetectIntrusion`, forwards `SensorTriggered(sensor)` to the `AlarmControlUnitActor`, where `sensor` carries the configured identity and zone. It never changes behavior.
 - **SirenActor**: Manages two behaviors, `sounding` and `silenced`, toggled by `Start` and `Stop` respectively. Starts in `silenced`.
 - **AlarmControlUnitActor**: The system's central state machine, implemented via behavior switching across five states: `disarmed` → `exitDelay` → `armed` → `entryDelay` → `alarm`. Timeouts (`ExitTimeout`, `EntryTimeout`) are scheduled via `withTimers` and arrive as regular mailbox messages, so a valid `PinEntered` processed before a timeout cancels the pending transition. Sensors are only reacted to in `armed` mode, and only if their zone is active.